



GitHub Actions: The Big Picture

Course Overview

Course Overview

In a world driven by innovation and continuous integration, understanding the basics of automation with GitHub Actions is essential for every developer who works on GitHub. In this course, GitHub Actions: The Big Picture, you'll understand the basics of GitHub Actions and the foundational aspects of uncovering its core concepts, advantages, common applications, and initial setup. You'll explore the essential components of an Actions workflow file from event triggers, steps, runners, jobs, and how everything works together. You'll gain hands-on experience in creating a basic workflow file that will print a message to the console log based on a specific event trigger. Next, you'll work with a basic application and use GitHub Actions for continuous integration and continuous deployment and see how you can build, test, and deploy an application to GitHub Pages. Whether you're an experienced developer seeking to streamline your workflows or a newcomer eager to embrace the power of automation, this course offers a holistic approach to integrating GitHub Actions seamlessly into your development routines. By the end of this course, you'll have a big picture knowledge and practical skills to revolutionize your development workflows through GitHub Actions, setting the stage for enhanced efficiency and productivity in your projects.

Getting Started with GitHub Actions

Overview of GitHub Actions

Automation in software development isn't just about speeding up processes. It's about elevating the overall quality of work, ensuring safety, and increasing team satisfaction. By implementing modern continuous integration and continuous deployment or CI/CD practices, teams can significantly reduce human errors, maintain consistent testing standards, and free up developers to focus on more impactful tasks. Embracing CI/CD automation isn't just beneficial. It's vital for agility and growth. And with the inclusion of GitHub Actions, the process becomes even more seamless and accessible, enabling you to automate all of your workflows within your GitHub environment. GitHub Actions automates the way you build, test, and deploy your code directly from your GitHub repositories. And at its core, there are three key benefits to using GitHub Actions. The first is increased efficiency. GitHub Actions allows you to automate



repetitive tasks, reducing manual effort and minimizing errors. By automating such tasks as code testing and deployment, you can focus more on writing code and less on managing the development process. The second, and this may seem obvious, but it's the integration with your GitHub repositories. GitHub Actions seamlessly integrates with your GitHub repositories, enabling you to define workflows directly within your code base on GitHub. This tight integration ensures that your workflows are version controlled and easily accessible to your team with all of the other great features that GitHub offers. And third is the flexibility you have in creating custom workflows. With GitHub Actions, you have the flexibility to define custom workflows tailored to your specific requirements. So, whether you're building a simple CI/CD pipeline or orchestrating complex, multi-step workflows, GitHub Actions can help you automate all of them.

Common Use Cases of GitHub Actions

GitHub Actions is incredibly versatile and can be used for a wide range of automation tasks. For example, some common use cases that I use within my team at GitHub is to use GitHub Actions for notification management. We use it to notify team members or external systems when specific events occur, such as the creation of a new issue, a pull request, review request, or a deployment status update. We also use GitHub Actions on a schedule to send custom notifications to Slack, Microsoft Teams, and other communication channels to keep the team informed about weekly standup discussions and other team-related information. Some other common use cases include issue management, pull request automation, repository maintenance, and community engagement. Now these are just a few examples of the automation tasks that you can customize with GitHub Actions. But beyond these automation tasks and common use cases, a selling point of GitHub Actions is using it for continuous integration and continuous deployment. Continuous integration is a software development practice where code changes are automatically built, tested, and integrated into the main code base frequently. GitHub Actions makes implementing CI pipelines seamlessly by allowing you to define workflows that automatically trigger on such events like a create, a push, or a schedule just to name a few. With GitHub Actions CI, you can automate testing by running automated tests against your code base to ensure that new changes don't introduce bugs or regressions. You can perform code quality checks by using linters, static code analysis tools, and other quality assurance checks to maintain code quality standards. You can build artifact generation, automatically build and package your application or library into deployable artifacts ready for further stages in the deployment pipeline with continuous deployment, which is the process of automatically deploying code changes to production or staging environments after passing through the CI pipeline. GitHub Actions enables you to define CD workflows that deploy your application or service seamlessly. With GitHub Actions CD, you can automate



deployment and deploy your application to various environments, such as staging or production, based on predefined conditions or those event triggers. You can implement rollback mechanisms to revert the previous versions in case of deployment failures or issues. And you can do environment provisioning and automatically provision and configure infrastructure required for deployment, such as virtual machines, containers, or serverless functions.

Benefits of GitHub Actions over Other CI/CD Platforms

Now, if you have experience with other popular CI/CD platforms like Azure DevOps, CircleCI, GitLab, Jenkins, Travis CI, Bamboo, or Bitbucket, they all have their own spin on providing comprehensive CI/CD solutions. But GitHub Actions offers a unique experience that provide several benefits for developers and teams that work on GitHub. The most notable benefit, and this comes as no surprise, is the tight integration with GitHub. GitHub Actions is tightly integrated with GitHub repositories, making it seamless to automate CI/CD workflows directly within the GitHub ecosystem. Now this integration simplifies setup and management, as well as providing a familiar interface for developers. There is no third-party integration needed. GitHub Actions are built into every GitHub repository. The second is creating and managing your workflow as code. With GitHub Actions, CI/CD workflows are defined using YAML syntax. These files are stored alongside the code in your GitHub repository. This allows you to version control your workflows along with your code, facilitating collaboration and ensuring consistency across environments. Another benefit is the vast library of actions. GitHub Actions provides a vast library of prebuilt actions that cover wide range of tasks from building and testing to deploying applications. This extensive library saves time and effort by allowing you to leverage existing actions rather than recreating an existing action. Third-party actions not only decrease code repetition, they also let you take advantage of the wider GitHub developer ecosystem. Why write custom integrations when they already exist? The best place to view these existing actions is to visit the GitHub Marketplace. The GitHub Marketplace is a central location where you can discover and share GitHub Actions. You can browse through a wide variety of Actions across different categories, such as CI/CD, code quality, deployment, and much more. Now each action listed in the marketplace typically includes a description, documentation on how to use it, list of contributors, stars, and a badge to indicate if a particular action is from a verified creator. GitHub Actions also offers flexibility in customization options, allowing you to define complex workflows with conditional logic, matrix builds, and other advanced features. This enables teams to tailor their CI/CD pipelines to their specific requirements and then integrate with third-party tools and services. You also have scalability. GitHub Actions is designed to scale with projects of all sizes, allowing you to run parallel jobs across multiple virtual environments to speed up workflow execution. Additionally, GitHub provides distributed runners hosted by GitHub or



allows you to set up self-hosted runners in your own environment for added scalability and control. Another benefit is that GitHub Actions are relatively cost-effective. GitHub Actions provide generous free usage limits for all public repositories and includes a certain amount of free minutes for private repositories. GitHub plans also offer free usage up to a certain limit depending on your plan choice. This makes it a cost-effective option for teams and projects of all sizes, especially if you're already using GitHub. So, now that we have a basic understanding of what GitHub Actions are and how they can be useful, let's build a simple GitHub Actions workflow together, and I'll explain the basic components and methods along the way.

Build a Workflow with GitHub Actions

Understand the Components of the Workflow File

The best way to understand how to use GitHub Actions is to see it in action and create a GitHub Actions workflow file. But before we dive in and create a basic workflow file, let's first understand the components within that file. An Actions workflow is a configurable automated process that is made up of one or more jobs defined using YAML syntax. These workflows follow a specific structure and are stored in the `.github/workflows` directory of your GitHub repository. So, let's break down the main components of a workflow file. The first component we see is name. The name component is a descriptive title for your workflow and can really be anything that you want. It's a way to give your workflow a recognizable identity, making it easier for you and others to understand its purpose. So, for example, you might call it continuous integration to indicate that it's responsible for automating code integration tasks. The next thing we see is the `on` component, which is setting the workflow's trigger event. This part specifies the GitHub events that trigger the execution of your workflow. It's like setting up the conditions under which your workflow should run. And you can choose from a variety of events, such as pushes to the repository, pull requests, issue comments, and much more. This flexibility allows you to tailor your workflow to respond to specific actions within your repository. After we define the trigger events, we see the section that defines the jobs. Now by default, jobs function independently and run in parallel. But you can also configure a job to wait for another job or jobs to complete before it begins. For instance, you could have separate build jobs for different architectures running concurrently with a subsequent packaging job that only initiates once all jobs have successfully concluded. The next thing we need to do is to specify a runner. Now runners define the execution environment in which your jobs will run. They determine the operating system, software dependencies, and hardware resources available to your workflow. Now, there are two basic types of runners available, GitHub-hosted runners and self-hosted runners. GitHub provides a variety of preconfigured runners for common use



cases, including Ubuntu, macOS, and Windows environments. Additionally, you can use self-hosted runners to customize the execution environment according to your specific requirements. A self-hosted runner is a system that you deploy and manage on your own infrastructure to execute GitHub Actions. Runners ensure consistency and provide the necessary infrastructure for your workflow to run successfully across different platforms. Now, after we specify the runner, we need to provide the steps. Steps are the individual actions that make up a job. They define the specific tasks that need to be executed to complete the job successfully. Each step represents a command, a script, or action to be performed, such as checking out code from your repository, installing dependencies, compiling code, running tests, or deploying artifacts. Steps are the building blocks of your workflow and allow you to break down complex tasks into smaller, manageable units. Within a step component, there are several optional keywords that you can specify to perform tasks or add clarity. You can check out GitHub's documentation for a full list of keywords available. But some of the common ones are `name`, which provide a descriptive name for the step to help identify its purpose within the workflow. So for example, `checkout` or `run` tests. You can also have `uses`, which specify an action to be used in the step. Here, we use `actions/checkout@v4`, which specifies that this step will use `@v4` of the `checkout` action that allows your workflow to access the contents of your repository so that you can run scripts or other actions against it such as running build and test tooling. In this step, we use the `actions/setup-node@v4` action to install the specified version of the Node.js. Now this example uses version 20 and puts both the `node` and `npm` commands in your path. We then use the `run` keyword to tell the job to execute a command on the runner. In this case, we're using `npm` to install the `bats` software testing package. And to finish this example, we run the `bats` command with the parameter that outputs the software version. These actions are reusable units of code that perform specific tasks. You can use actions defined in the same repository as the workflow file or from other repositories. For instance, you might use an action to publish artifacts or deploy your application. Now you also have `run`, which allows you to specify shell commands to be executed directly in the step. This can include a running scripts or executing custom commands. For example, you might run a shell script to build your project or execute a command to install dependencies. In this example, we're using `npm` to install the `bats` software testing package. And by utilizing these components effectively, you can create custom and robust workflows to automate various tasks within your GitHub repository. Each step plays a crucial role in executing specific actions or commands as part of the overall workflow process.

Create a Basic Actions Workflow File

Now that we have a basic understanding of the components within a GitHub Actions workflow file, let's see how this would work in a demo repository. So if you



want to follow along, you can navigate to the template repository for this course at github.com/a-a-ron/github-actions-big-picture. Now you'll need to create a copy of this template repository. It contains an example web application that will deploy the GitHub Pages using GitHub Actions. This template repository also contains copies of all of the files created during this course in their completed state if you want to view the end results. These files are located in the `demo-files` directory. Now you can follow along by clicking the green `Use this template` button located in the top right and then select `Create a new repository`. You'll need to fill out the form by selecting you as the owner, give the repo a name and a description, and then make sure to set the default visibility of the repo to public as private repositories use Actions minutes, while public repositories can use GitHub-hosted runners for free and then create the repository. Now, you have two ways to create a new workflow. You can create a new YAML file in the `.github/workflows` directory or from the Actions tab of your repository. The first way is pretty simple and straightforward. You can create a new file in the correct directory, give it the YAML extension, and then build out your workflow. But the second way has some nice advantages. So, let's see what GitHub gives us. By clicking on the Actions tab at the top of the repository, GitHub has suggested several starter workflows based on the languages of the code in this repository. Since this repository has JavaScript and Node.js files, the suggestions GitHub provides are common workflows used in those languages. The suggestions are also organized into categories, such as deployment, security, continuous integration, automation, and pages. And you can also search for more workflows using the search bar at the top. But for this demo, let's skip these prebuilt workflows and create one from scratch. So at the top, let's select the option to set up a workflow for ourselves. This will give us a blank workflow file already in the `.github/workflows` directory. So let's change that default name from `main.yml` to `hello-world.yml`. With our workflow file in place, let's create a workflow that automatically prints out a friendly hello world to the actions logs whenever someone pushes to the repository. Now, thinking through the workflow components that we just talked about earlier, we need to provide a name, event trigger, a job, a runner, steps to occur within a job, and a specific action to run within a step. So let's get started with the first component and give our workflow a name. Now, since we're printing hello world with this workflow, let's call it hello-world. Each workflow file must incorporate at least one triggering event to initiate its execution. Now again, these events encompass various GitHub Actions, ranging from pull request creation to issue commenting and even the successful execution of another workflow. And also, workflows can be externally activated through API calls using the repository dispatch event. Now these events facilitate seamless integration between your CI and CD pipelines and the development platform, offering you the flexibility to interact with other tools. So in our hello world example, let's do something simple and configure the workflow event to trigger on a push event. These push events trigger the workflow whenever you push a commit or tag or create a repository from a template. So once we've completed writing this `hello-world.yml` file and commit



it, the workflow will be activated on a push event. But before that, we need to define what we want to happen on these push events. We can do this by defining a job. Each workflow has one or more jobs, each composed of a sequence of interconnected executable steps. So, we'll start the jobs section of our workflow by naming our job `hello-world-job`. Next, we need to specify a runner. A runner functions as the engine that executes the job within a workflow. There are primarily two types of runners, again, GitHub-hosted runners and self-hosted runners. A self-hosted runner is a system that you deploy and oversee on your infrastructure to carry out GitHub Actions jobs. And in contrast, a GitHub-hosted runner provides a newly provisioned virtual machine operating on Ubuntu Linux, Microsoft Windows, or macOS. So if you decide to use a GitHub-hosted runner, you have the option to specify the desired OS version or choose `-latest` for the most recent stable version. Additionally, GitHub offers larger runner versions with enhanced features such as runner groups for organizational access control. And GitHub-hosted runners are generally recommended for their simplicity and reduced administrative overhead, but self-hosted runners could be a good choice based on your situation. So for our example, we'll use `ubuntu-latest` on our GitHub-hosted runner. The next thing we need to do is add a step to run our first action. As mentioned before, each job is made up of one or more steps. And each step can either run a script that you define or run in action, which is a reusable extension that you can simplify your workflow. So in our hello world workflow, let's use the common checkout action. This action enables your workflow to retrieve the contents of your repository so that you can run scripts or other actions against it. This is really useful for tasks like running build and test tooling. Incorporating the checkout action is essential whenever your workflow requires access to the repository's contents. Let's define this using the steps and then the optional name with the description of something like `check out repository code`. And then below that, we need to type `uses: actions/checkout@v4`. This specifies that this step will use version 4 of the checkout action. Similar to any dependency, you should indicate the version of the action that you like to use based on your comfort with automatically accepting updates to the action. You can find the latest version of an action by finding the action in GitHub Marketplace. If you don't specify a version, the latest version will automatically be used and could break your workflow when breaking changes are made to the action. The last thing we'll do is use a step to run a bash script that will print out our contents that we have in a file called `hello_world.txt` within the same repository. So now that we have a workflow file with the name, a trigger event, a job, a runner, and some steps to perform some actions, let's commit these changes to our GitHub repository and see what happened. After our commit, let's navigate to the Actions tab at the top of our GitHub repository. Here, we see a workflow that is running for hello-world. And after a few seconds, the workflow should complete. And if successful, we should see a green checkmark next to the workflow. And checking on the workflow, we see our hello world job that ran within our workflow. Let's check the job and see what happened. We see information on setting up the job. This is where we see information on our runner,



which is Ubuntu. We also see the step to check out our repository code using the checkout action with version 4. And we see the step to print out the contents from our hello-world.txt file that says Hello! And congrats, you've successfully built and run your first GitHub Actions workflow file. Now, as long as we did everything correctly, we should see all of the jobs listed out on the screen with the time it took to run each one alongside it. Now if you run into an error, make sure that you formatted the contents of your YAML file correctly. YAML formatting relies heavily on whitespace, so any discrepancies may lead to errors. Now that we've learned about the basic building blocks of GitHub Actions, let's use them to do something a bit more interesting and useful than writing a Hello World message to the actions log. So let's write a simple CI/CD workflow to build, test, and deploy a web application to GitHub Pages using GitHub Actions.

Build, Test, and Deploy an Application with GitHub Actions

Build an Application with GitHub Actions

In the previous module, we went over the basics of GitHub Actions by making a simple hello world workflow that printed the contents of a file to our Actions logs and then going over how each component in the workflow file works. So now, let's build upon that knowledge and dive into something a little bit more hands on. Let's look at something more practical by creating a CI/CD workflow file and see how we can build, test, and configure and then deploy a simple web application to GitHub Pages, all using GitHub Actions. We'll be using the same example repository as before. So feel free to copy my actions and follow along. However, keep in mind that this course is a big picture course of using GitHub Actions. So I won't spend a lot of time explaining in detail everything that I do. And I'm going to cover these steps a little fast. The purpose is to get a big picture understanding of what GitHub Actions are, how they work, and how you can use them in your own development workflow. So if you'd like, sit back, relax, and watch the demo. Otherwise, feel free to pause along the way to follow along. And with that, let's get started. Using the same repository as before, let's click on the Actions tab located at the top of the repository. And then from there, let's click on the New workflow button. As we saw before, GitHub recommends some starter workflows based on the contents of the files within our repository. But again, for this example, let's set up our workflow by ourselves. Notice that our workflow file is correctly located in the .github/workflows directory. Now this will automatically happen when we create a new workflow through the Actions tab. So, let's change the name from main.yml to build-test-deploy.yml. This will give us a good idea of what this workflow is



going to do. And as we did earlier, we start our workflow with a name. We'll call it build-test-deploy. Let's set the event trigger to push. We do this with the on value and then the event type that we want. Next, we needed to define the jobs that we want to occur within this workflow. We do this with the jobs value, and then we can define our first job. Let's give it a name of build since this will be our first job in our build, test, and deploy phase of our application within this workflow. Next, we need to define a runner within each job. So looking forward, we plan to have three jobs in this workflow, a build, test, and deploy. So, we'll need to find a runner for each of these jobs. To do this, we use the runs-on value and then we can specify a runner. So let's use ubuntu-latest to use the latest version. Next comes steps within our build job. We define this with the steps value and then the specific actions we want to occur. Since we want this job to reference code within this repository, we need to use the checkout action. This action retrieves the repository's code and allows our workflow to access the contents of our repository. We'll give it a name for good practice, something like checkout repo, and then define the action with uses and then actions/checkout@v4. Now, since we're using Node.js as our build tool, we'll need to set it up using the setup node action. To do this, we'll need a step similar to the previous one. Let's give the step a name of use node.js, and then let's define the action we want with actions/setup-node@v4. Since we want to use a specific version of Node.js, we need to define that within our step. We can look at the usage instructions for this particular action by going to the GitHub Marketplace and looking up this action or by going to its GitHub repository and reading the README. And after review, we see that we can do this with the with value and using node-version and then defining the version we want. So for this example, we'll use node version 20. The setup node action just like the checkout action is among the many actions that you can find on GitHub Marketplace. Now the next thing we need to do is to build the code. Now we have our build tool set up and ready to go. We'll use npm to build our Next.js application. You can use the same commands that you use on your local machine to build your code. So we'll simply add the following run commands to install npm and then run its build. We can do this by using run and then our first command of npm install and then our next run command of npm run build. Since our workflow runs on push events, any pushes to the repository, including when we commit this file to the repository, will trigger the workflow. So, let's commit our changes. Let's now navigate back to our Actions tab, and we see two workflows running. Remember, our hello world workflow is still triggered with all push events just like our new build, test, and deploy workflow. So let's disregard our hello world workflow and click on our build, test, and deploy one. Let's click on our build job and then select the run npm run build step. And nice, here are the details of our build. So, now that we've built our application, it's important to verify that it meets our expectations. And thanks to Actions, we can automate various tests, eliminating the need to do them all manually. So next, we'll enhance our workflow by integrating a basic test.



Test the Application with GitHub Actions

Now that we have our build job for our Next.js web application, our next job involves verifying its functionality through testing. To ensure everything aligns with our expectations, let's create a basic unit test. Now it's worth noting that actions have the capability to execute various types of tests beyond just simple unit tests. But for this course, we'll stick to a basic unit test. So navigating back to our workflow file, let's add another job just after our build job. Now we can type everything out starting with the name, runner, and each of the steps. But our test job will be very similar to our build job, but with a few changes. So let's copy our build job and paste it below, and then let's make the needed changes. Let's first change the name to say test instead of build, and then we need to add something new. Since it only makes sense to run a test on a successful build, we can define this in our job by including the `needs` keyword followed by the name of the job that we want, which is `build`. The `needs` keyword enforces that jobs run in a sequential order where the current job is contingent upon the success of another job. This comes up a lot. When you think about CI and CD. You don't necessarily want to test your application if it can't successfully build. And the same goes for deployments. You don't want to deploy your application if it fails its tests. We'll see this again when we create our deploy job. The last thing we need to do is change the last line in our test job. Instead of `npm run build`, let's put `npm test`. Now, most of this should look pretty familiar as it's almost identical to the build job above it. We named our job `test`, designated a GitHub hosted runner to use `Ubuntu latest`, and used the same prebuilt actions to check out our code and then install `npm`. Now, you may be wondering why we needed to define a lot of the same things again. Well, it's because jobs are completely independent. So steps are scoped locally and need to be repeated for every job. Now, while this may seem like a hassle, it also means that you can make different decisions for each job, such as choosing a different runner or a different version of any component according to your needs. And just like in the build job, we set up `Node.js` and then installed it. But now, rather than running `npm build`, we're going to run `npm test` to test out the code just as we would from the command line. So, this looks good. Let's commit these changes to our repository and then navigate back to the Actions tab. Let's click on our new workflow run, and notice that we see both the build and test jobs are visually linked together, implying that one needs the other. So let's click on our test job and then select the `run npm test` step to see our test. It looks like our test ran successfully and performed some linting using `ESLint`. And that's it. It's that easy to automate testing with GitHub Actions. With just a few steps in our workflow file, you can build and test an application. Now, of course, there is much more to testing than running a unit test in a singular environment. But hopefully, this gives you a starting point of what you can do. So next, let's configure our deployment environment so that it will be ready when we deploy it.



Configure the Deployment Environment

Now that we've gone through the building and testing phases with GitHub Actions, the spotlight is about to shift to deployment. However, before taking that leap, it's crucial to establish and fine-tune a deployment environment. And these environments aren't just about directing deployments to specific destinations for various purposes. They also serve as a single location for tracking deployment-related information. So, let's navigate back to our GitHub repository and locate the Settings tab. In the left side bar, select Environment and then click on New environment. Deployment environments are commonly named deployment, staging, or production to identify where they fit in the software delivery lifecycle. So we'll keep it simple for now and name our environment Production. And then we can click on Configure environment. You'll notice that there are several things that we can do here apart from providing a clear path for deployment. Environments come with a range of features aimed at ensuring their security and integrity. For instance, you can set up restrictions on which branches or tags are allowed to be deployed to a particular environment. Additionally, you have the option to store environment-specific variables and secrets securely. You can establish protection rules, such as mandating reviews from specific individuals or teams before any deployment takes place. But for the time being, let's stick with the default settings. But as you can see, you can quickly start to customize your environment. Before we move on to our deployment part of the workflow, let's enable GitHub Pages, which is what we'll use as our deployment target. We can do this by selecting Pages from the left sidebar. Under the Build and deployment section, select the drop-down source to display GitHub Actions. Now that our deployment environment is configured, we can move on to the deploy stage by adding our deploy job and then deploying our application to GitHub Pages.

Deploy the Application to GitHub Pages

With GitHub Pages enabled and our environment created and configured, we're finally ready to add our deploy job. Let's navigate back to our build-test-deploy.yml file in our .github/workflows directory and add the job. Now, for the sake of repetition and time, instead of coding each line in this demo, I'm going to paste in the already completed deploy job, and then we can look at what's going on. We've named our job as deploy. And just like our test job needing the build job to succeed before it can run, our deploy job needs the test job to successfully run before it will run. Below that we have something new. Here we define our permissions. Within Actions, the permissions keyword is like the gatekeeper, determining what our workflow is allowed to do. Now getting these permissions just right ensures smooth interaction between our workflow and different areas of our GitHub repository. Now for our particular task, we're



tweaking permissions for the GitHub token. Think of this token as the master key. It grants our workflow the power to access and tweak various parts of our repository. And since we're gearing up for a Pages deployment, it's crucial that our token has the green light for write access. Next, we need to define our deployment environment. In the previous video, we set up a production environment. Now as we gear up to deploy our application, we need to pinpoint that environment as our deployment destination. We achieve this by passing two parameters to the environment, the name of the target environment and the URL of the deployment. When it comes to the URL, we're using `steps.deployment.outputs.page_url` to snag the URL from the final deployment step. Now, don't worry if this sounds a bit technical. I'll walk you through exactly how we found this URL in just a moment. Oh and while we're on the subject, we're sticking with our trusted GitHub hosted runner with `ubuntu-latest`, just like before. After this, our first step is to check out the repository with authentication. And just like your other build and test tasks, our first step is to employ the checkout action, fetching all of the content from our repository. However, this time around, we're not just here for a read. We're gearing up to do some writing too. So it's crucial to make sure that we're properly authenticated with GitHub to unlock those permissions. So here's where the magic happens. The `token secrets.GITHUB_TOKEN`, it's like the golden ticket provided by Actions for authentication purposes. When coupled with the permissions we've just set up, it guarantees our workflow has all of the necessary access rights to get the job done. Next, we set up the node environment, again for the deploy job. Again, remember, every job runs independently, so you need to repeat these steps for each job. And after this, we config GitHub Pages for deployment. Now, before we dive into building our application, we prep it for Pages deployment using the `configure-pages` action. We make sure to inform the action that we're dealing with the Next.js static site by specifying `static_site_generator: next`. This helps ensure everything gets set up just right for our Next.js project. And after this, we run `npm install` and `npm build` and then a section to upload artifacts to pages. In the world of CI and CD, think of an artifact as a neat little package or collection of files that comes out of our build process, ready to be shared or deployed. Now in this step, we're using the `upload-pages-artifact` action to wrap up the static site created by Next.js during the `npm run build` phase. So, how do we do it? Well, we just point the action to the right place, by specifying `path` out. Then, just like a trusty delivery service, the `upload-pages-artifact` action swoops in and sends our artifact straight to Pages for deployment. The last step is to deploy our application to Pages. The `deploy-page` action swoops in to do the heavy lifting. It takes those tidy artifacts we've prepared and uploaded in the previous steps and sends them straight to Pages for deployment. Now, by assigning an ID deployment to this step, we've given it a unique label. So, why does this matter? Well, it comes in handy if we need to refer back to this step in future tasks or keep an eye on its progress in more complex workflows. Remember when we set up the environment a few steps back? We used this identifier to snag the



URL of our site using `url.steps.deployment.outputs.page_url`. Once this step wraps up successfully, our Next.js static content will be live and ready to view on GitHub Pages. So let's commit these changes to our GitHub repository and return back to our Actions tab. Let's find and click on our workflow run. And then we can see our jobs running. Notice that the build job needs to succeed before our test job can run. And after our test job completes successfully, our deploy job can then run and deploy our application to our production environment. And once that is completed, we can click on our deployment URL and see our deployed action.

Summary

Nice job. You've successfully set up an automated workflow with GitHub Actions to deploy a basic Next.js application to GitHub Pages. Now that we've covered the basics of writing workflow files and creating a simple CI and CD pipeline, it's time to explore workflow automation at scale. You've gained an understanding of how to build, test, and deploy applications using Actions and GitHub Pages. Now you're ready to start building your own custom CI/CD pipelines using existing actions. And as you scale your projects, the power of automation will become increasingly evident. The next steps in your learning journey can involve diving deeper into custom GitHub Actions for your specific needs. This can include leveraging more advanced features and integrating third-party workflows from the GitHub Marketplace, as well as writing custom scripts to extend the capabilities of GitHub Actions. Hopefully, this course gave you a big-picture understanding of what GitHub Actions are and how you can use them in your own software development workflows. So, thanks for taking this course, and I'll see you in the next one.